



INTE130 – Object-Oriented Programming Assignment.

(CyberShield system Project)

Submitted Parts: Part C

Prepared by :

Sultan Khalid Al Yarabi - 2318592

Malik Marzooq Al Rahbi - 2527871

Khamis Hamed Alowaisi - 2422670

Mohammed Naeem Al Gheilani – 2526168

Instructor Name:

Dr. Akbar khanan

Semester: Spring 26

Part D – System implementation

Introduction

This implementation is based on the cybersheild in part C. The system use Python and OOP concepts to create a connected multi-agent cybersecurity system.

The purpose of the implementation is to show how the founder and the digital agents are workes together inside the system.

1. System implementation Concept

The implementation starts when the founder creates a cybersecurity task like checking system security.

Then the task send to three digital agents:

Threat Agent

This agent is specialized in detecting subspecies activity on the system.

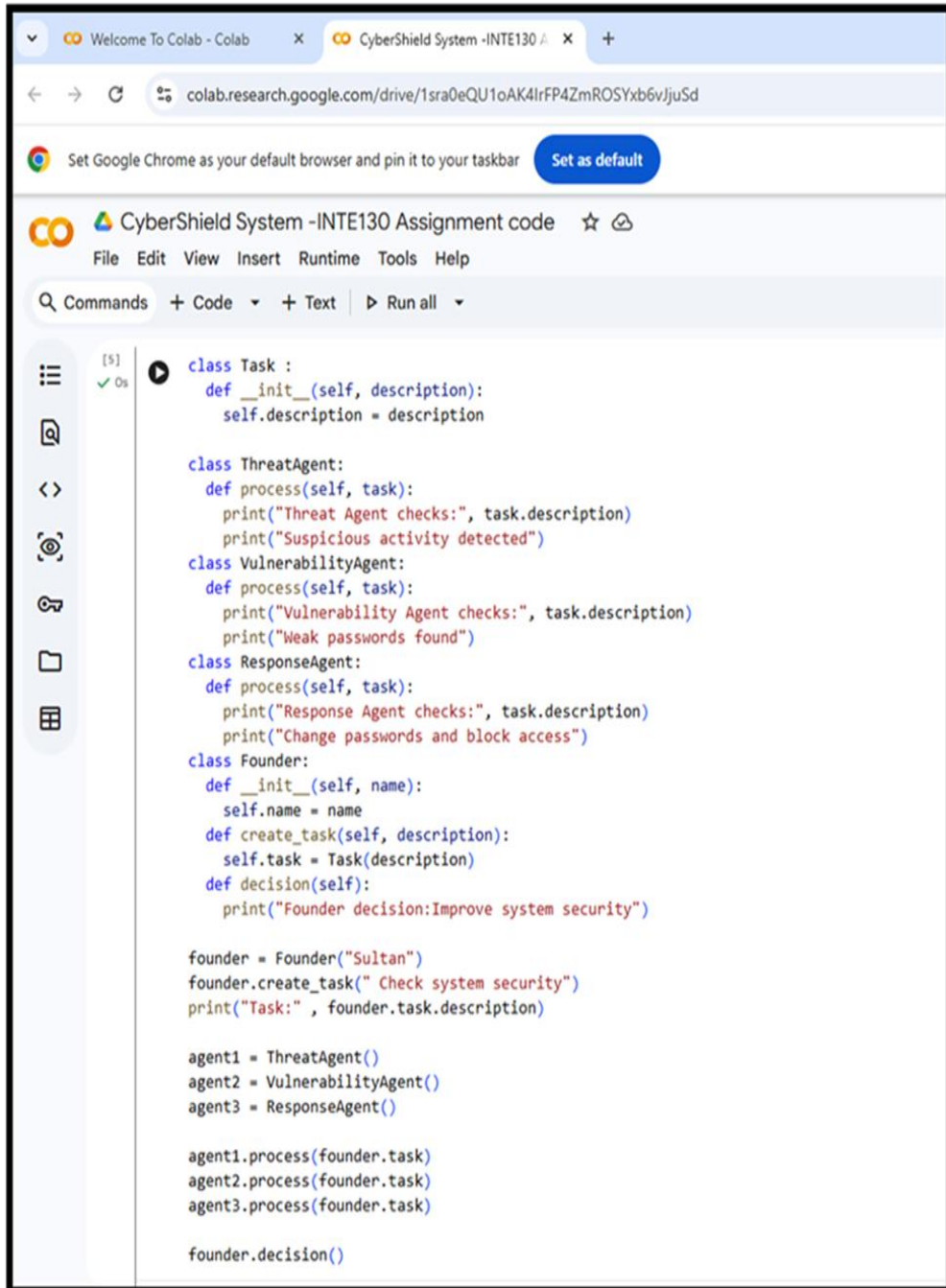
Vulnerabilities Agent:

In this agent system will be checked for any weakness or things that will make threats easy.

Response Agent:

In this agent the system cybershield will give you recommendations and solutions that will help you for any vulnerabilities found.

Finally the founder will have the full report fill with detections, problems and recommendations that the system gave about the task, and he will have the responsibility to make the appropriatdecision about the task.



colab.research.google.com/drive/1sra0eQU1oAK4lrFP4ZmROSYxb6vjjuSd

Set Google Chrome as your default browser and pin it to your taskbar Set as default

CyberShield System -INTE130 Assignment code

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[5]
✓ Os
class Task :
    def __init__(self, description):
        self.description = description

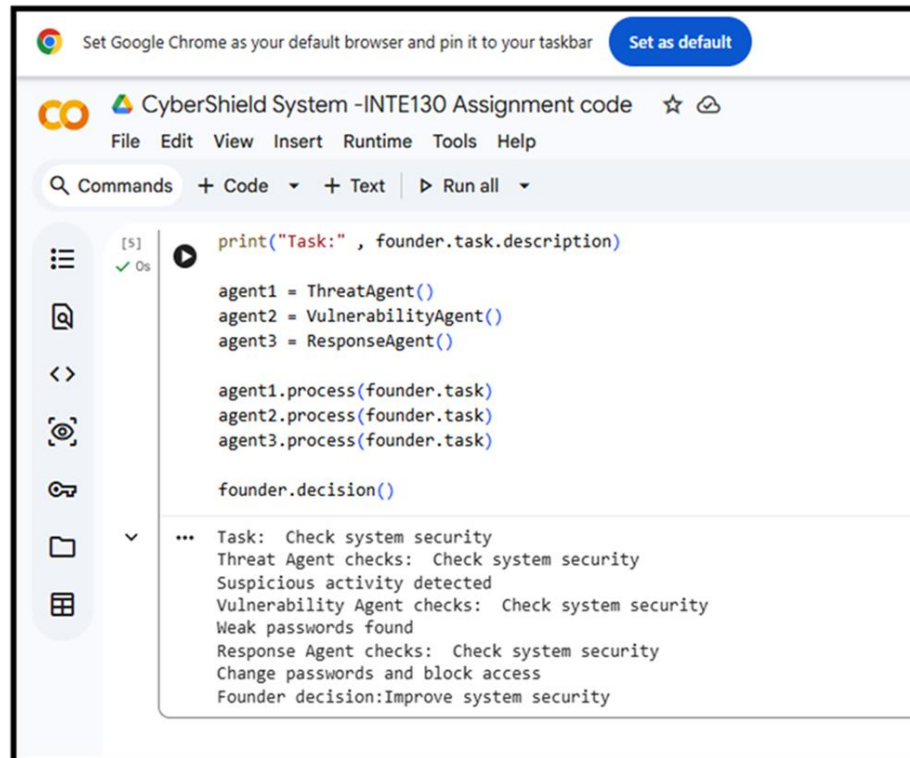
class ThreatAgent:
    def process(self, task):
        print("Threat Agent checks:", task.description)
        print("Suspicious activity detected")
class VulnerabilityAgent:
    def process(self, task):
        print("Vulnerability Agent checks:", task.description)
        print("Weak passwords found")
class ResponseAgent:
    def process(self, task):
        print("Response Agent checks:", task.description)
        print("Change passwords and block access")
class Founder:
    def __init__(self, name):
        self.name = name
    def create_task(self, description):
        self.task = Task(description)
    def decision(self):
        print("Founder decision:Improve system security")

founder = Founder("Sultan")
founder.create_task(" Check system security")
print("Task:" , founder.task.description)

agent1 = ThreatAgent()
agent2 = VulnerabilityAgent()
agent3 = ResponseAgent()

agent1.process(founder.task)
agent2.process(founder.task)
agent3.process(founder.task)

founder.decision()
```



2. Python code explanation:

There are several classes in the implementation:

- a. Task Class:
Stores the task description
- b. Threat Agent class:
Checking for any unwanted activity
- c. Vulnerability Agent Class:
Detect weaknesses and vulnerabilities on task
- d. Response Agent Class:
Giving solutions and recommendations
- e. Founder Class:
Create class and make the final decision after review the final report.

3. System Workflow:

- a. Founder create task
- b. Task goes to Threat Agent and get processed
- c. Then the task will move to Vulnerabilities Agent
- d. After the weakness got checked, task will go to Response Agent and give the appropriate response
- e. Finally, the founder will have a full report about the system and make the final decision.

This project is mainly focused on using OOP in processing tasks like classes. Threat Agent, Founder and every other agents are basic classes that system use to enhance the task. It also includes methods and objects that make system more organized. And finally, data security is necessary so that encapsulation keep it inside where it is unreachable.

- ## 4. CyberShield implementation shows how one person can manage cybersecurity company using digital agents. System is connected, organized, and simple to understand. The implementation also demonstrates how Python and OOP concept can be used to build a practical multi-agent system